

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY



SOICT

**REALTIME NEWS TREND & SENTIMENT
ANALYSIS PLATFORM**

BIG DATA & PROCESSING

Supervisor: Dr. Tran Viet Trung

Group 12:	Nguyen Mau Trung	20225534
	Vu Trung Thanh	20220066
	Luong Huu Thanh	20225458
	Doan Anh Vu	20225465
	Nguyen Tien Thang	20225530

Hanoi, November 2025

Contents

1	Problem Definition	4
1.1	Selected Problem	4
1.2	Analysis of Suitability for Big Data	5
1.3	Scope and Limitations	6
2	Architecture and Design	7
2.1	Overall Architecture	7
2.2	Detailed Components and Roles	9
2.2.1	Ingestion Layer	9
2.2.2	Speed Layer	10
2.2.3	Batch Layer	11
2.2.4	Serving Layer	12
2.3	Data Flow and Interactions	12
2.3.1	Ingestion Layer: Data Source Integration	13
2.3.2	Speed Layer: Real-Time RSS Processing	13
2.3.3	Batch Layer: Historical RSS + GDELT Processing	14
2.3.4	Serving Layer: Query and Visualization Interface	15
2.4	Business Logic Formulas	17
3	Implementation Details	18
3.1	Source Code Documentation	18
3.1.1	Ingestion Layer - Data Source Integration	18
3.1.2	Speed Layer - Real-Time RSS Processing	20
3.1.3	Batch Layer - Historical Data Processing	22
3.1.4	Serving Layer - Query and Visualization Interface	26
3.2	Data Schemas	33
3.2.1	Avro Schema: <code>news_raw.avsc</code>	33
3.2.2	MongoDB Collection Schemas	34
3.3	Environment Configuration	35
3.3.1	Container Configuration	35
3.3.2	Environment Variables	35
3.4	Deployment Strategy	35

3.4.1	Kubernetes Deployment	35
3.4.2	Local Development with Minikube	37
3.5	Monitoring Setup	37
3.5.1	Pipeline Status Indicators	37
3.5.2	Dashboard Metrics	37
3.5.3	Data Lifecycle Management	38
4	Lessons Learned	38
4.1	Lesson 1: Handling Multilingual GDELT Data	38
4.1.1	Problem Description	38
4.1.2	Approaches Tried	38
4.1.3	Final Solution	39
4.1.4	Key Takeaways	39
4.2	Lesson 2: Logic Redundancy in Analytics Jobs	39
4.2.1	Problem Description	39
4.2.2	Approaches Tried	40
4.2.3	Final Solution	40
4.2.4	Key Takeaways	40
4.3	Lesson 3: Missing Categories in Real-Time Data	40
4.3.1	Problem Description	40
4.3.2	Approaches Tried	41
4.3.3	Final Solution	41
4.3.4	Key Takeaways	41
4.4	Lesson 4: The “Serving Layer” Gaps	41
4.4.1	Problem Description	41
4.4.2	Approaches Tried	42
4.4.3	Final Solution	42
4.4.4	Key Takeaways	42
4.5	Lesson 5: Airflow DAG Synchronization	42
4.5.1	Problem Description	42
4.5.2	Approaches Tried	43
4.5.3	Final Solution	43
4.5.4	Key Takeaways	43
4.6	Lesson 6: Minikube Resource Exhaustion	43

4.6.1	Problem Description	43
4.6.2	Approaches Tried	44
4.6.3	Final Solution	44
4.6.4	Key Takeaways	44
4.7	Lesson 7: “Silent” Data Failures	44
4.7.1	Problem Description	44
4.7.2	Approaches Tried	45
4.7.3	Final Solution	45
4.7.4	Key Takeaways	45
4.8	Lesson 8: Vertical vs. Horizontal Scaling for Spark	45
4.8.1	Problem Description	45
4.8.2	Approaches Tried	46
4.8.3	Final Solution	46
4.8.4	Key Takeaways	46
4.9	Lesson 9: Encoding Issues (UTF-8)	46
4.9.1	Problem Description	46
4.9.2	Approaches Tried	47
4.9.3	Final Solution	47
4.9.4	Key Takeaways	47
4.10	Lesson 10: Automated Data Lifecycle	47
4.10.1	Problem Description	47
4.10.2	Approaches Tried	47
4.10.3	Final Solution	48
4.10.4	Key Takeaways	48

1 Problem Definition

1.1 Selected Problem

In the modern digital media landscape, the volume and velocity of news content have grown exponentially. Millions of articles are published daily across thousands of sources, creating an overwhelming information stream that challenges both consumers and analysts. Traditional methods of news monitoring—manual curation, keyword alerts, and periodic reports—fail to keep pace with the 24/7 news cycle and cannot provide the real-time situational awareness required by businesses, researchers, and decision-makers.

Based on the business requirements outlined in this project, three core problems are identified.

Problem 1: Real-time sentiment monitoring lacks scalability and responsiveness.

Market analysts, PR teams, and researchers require immediate awareness of sentiment shifts in news coverage. Manual sentiment assessment is slow, subjective, and cannot scale to the thousands of articles published hourly. Without automated sentiment tracking, organizations miss early warning signals that could inform investment decisions, crisis response, or strategic pivots. For example, a sudden surge in negative sentiment around a company (e.g., from an average sentiment score of +0.4 to -0.2 within hours) may indicate an emerging PR crisis or regulatory concern that traditional monitoring would detect only after significant impact has occurred.

Problem 2: Category classification and trend detection are labor-intensive and inconsistent.

News articles span diverse topics—politics, technology, business, health, entertainment—but manual categorization is error-prone and cannot operate at scale. Analysts struggle to identify which topics are trending, which are declining, and how coverage patterns evolve over time. The system must compute trend indicators by comparing current article volumes and engagement against historical baselines. For instance, detecting a 300% increase in “Technology” articles mentioning “artificial intelligence” within a 24-hour window, compared to a weekly average, signals a significant trend event worthy of attention.

Problem 3: Historical analytics and real-time insights exist in silos.

Organizations often maintain separate systems for archival analysis and real-time monitoring, leading to fragmented insights. Batch analytics provide depth but lack freshness; stream-

ing analytics provide immediacy but lack historical context. A unified system must merge both perspectives, enabling queries that combine real-time sentiment (from the last 72 hours of streaming data) with historical baselines (from months of batch-processed archives). This dual-layer approach allows detection of anomalies that single-layer systems would miss.

In summary, the central problem addressed in this project is: **how to build a scalable, real-time news analytics pipeline that combines streaming ingestion with batch processing to deliver automated sentiment analysis, intelligent categorization, and trend detection for large-scale news data.**

1.2 Analysis of Suitability for Big Data

The three problems identified above require continuous processing of high-velocity news streams, systematic analysis of large historical archives, and scalable analytical workflows. These requirements align naturally with Big Data technologies, which are designed to handle streaming data, large-scale batch aggregation, and complex query-time analytics.

Suitability for Problem 1: Real-Time Sentiment Monitoring. Sentiment tracking at scale requires processing thousands of articles per hour with minimal latency. Big Data streaming frameworks enable continuous ingestion from multiple RSS feeds and news APIs, applying lightweight sentiment analysis as data arrives. Apache Kafka serves as a durable message buffer, decoupling ingestion from processing and enabling exactly-once semantics. Spark Structured Streaming provides windowed aggregations (e.g., 5-minute tumbling windows) for near real-time sentiment computation. This architecture supports the sub-minute latency required for actionable sentiment alerts while maintaining fault tolerance through checkpointing.

Suitability for Problem 2: Category Classification and Trend Detection. Large-scale text classification demands distributed computing resources that exceed single-machine capabilities. Big Data batch processing with Apache Spark enables parallel feature extraction (TF-IDF, word embeddings) and model training (Logistic Regression) across cluster nodes. Historical trend analysis requires aggregating millions of articles by category, source, and time period—operations that benefit from Spark’s columnar processing and in-memory computation. The system computes rolling statistics, burstiness metrics, and comparative baselines that would be infeasible with traditional database systems.

Suitability for Problem 3: Unified Historical and Real-Time Analytics. The Lambda Architecture paradigm addresses the fundamental tension between accuracy (batch) and freshness (streaming). Big Data architectures support query-time merging of batch-computed

historical baselines with streaming-computed real-time metrics. MongoDB serves as the unified serving layer, storing both batch analytics results and streaming-updated real-time views. This enables the Streamlit dashboard to display comprehensive insights that combine weeks of historical context with the last 24 hours of streaming data without maintaining separate query systems.

In conclusion, Big Data technologies provide the necessary foundation to address scalability limitations, processing latency requirements, and analytical complexity. By unifying streaming and batch processing within a Lambda Architecture, the system transforms raw news feeds into actionable intelligence that directly supports monitoring, research, and decision-making workflows.

1.3 Scope and Limitations

- **Scope:**

- **Real-time ingestion from multiple news sources:** The project continuously collects news articles from RSS feeds. Incoming data is processed in near real-time to extract metadata, apply sentiment scoring, and perform lightweight categorization. This enables the system to monitor current news activity within minutes of publication.
- **Batch processing of historical news archives:** Historical articles are collected from RSS feeds and the GDELT (Global Database of Events, Language, and Tone) API, processed in scheduled batch jobs to compute comprehensive analytics including machine learning-based classification, advanced aggregations, time-series statistics, and graph-based analysis of source-entity relationships. These batch results serve as reference baselines for real-time comparison.
- **Lambda Architecture for unified analytics:** The project integrates streaming results with batch-computed baselines to provide contextual insights. Recent news activity from the Speed Layer is compared against historical patterns from the Batch Layer to determine whether observed trends represent normal behavior, emerging topics, or significant sentiment shifts.
- **Interactive visualization dashboard:** The system provides a Streamlit-based dashboard for exploring news trends, sentiment distributions, category breakdowns, and keyword analysis. Both real-time streaming data and historical batch data are

visualized to support comprehensive situational awareness.

- **Limitations:**

Simplified sentiment analysis: Sentiment analysis uses keyword-based scoring for the Speed Layer (to minimize latency) and lightweight MLlib classifiers for the Batch Layer. While efficient for large-scale processing, this approach may not capture nuanced sentiment, sarcasm, or context-dependent expressions with high accuracy.

Restricted data sources: The project focuses primarily on English-language news from RSS feeds and GDELT. Social media platforms (Twitter, Reddit), paywalled content, and non-English sources are not included, which limits the completeness of news coverage representation.

Language constraints: The analysis assumes English-language content. Articles in other languages are filtered during ingestion to ensure consistent NLP processing. Multilingual sentiment analysis and cross-language trend detection are outside the scope of this project.

Local development environment: The Kubernetes deployment is designed for Minikube (local development) rather than production cloud infrastructure. Resource constraints limit the number of concurrent services and the scale of data that can be processed in a single development session.

No business outcome evaluation: The project demonstrates technical feasibility and analytical capabilities but does not include quantitative evaluation of business impact such as decision improvement, accuracy, or ROI metrics.

2 Architecture and Design

2.1 Overall Architecture

The system is designed following the **Lambda Architecture** paradigm, which combines batch processing and stream processing to support both **historical accuracy** and **low-latency analytics**. This architectural model is particularly well-suited for news analytics, where large volumes of historical articles coexist with high-velocity real-time news streams.

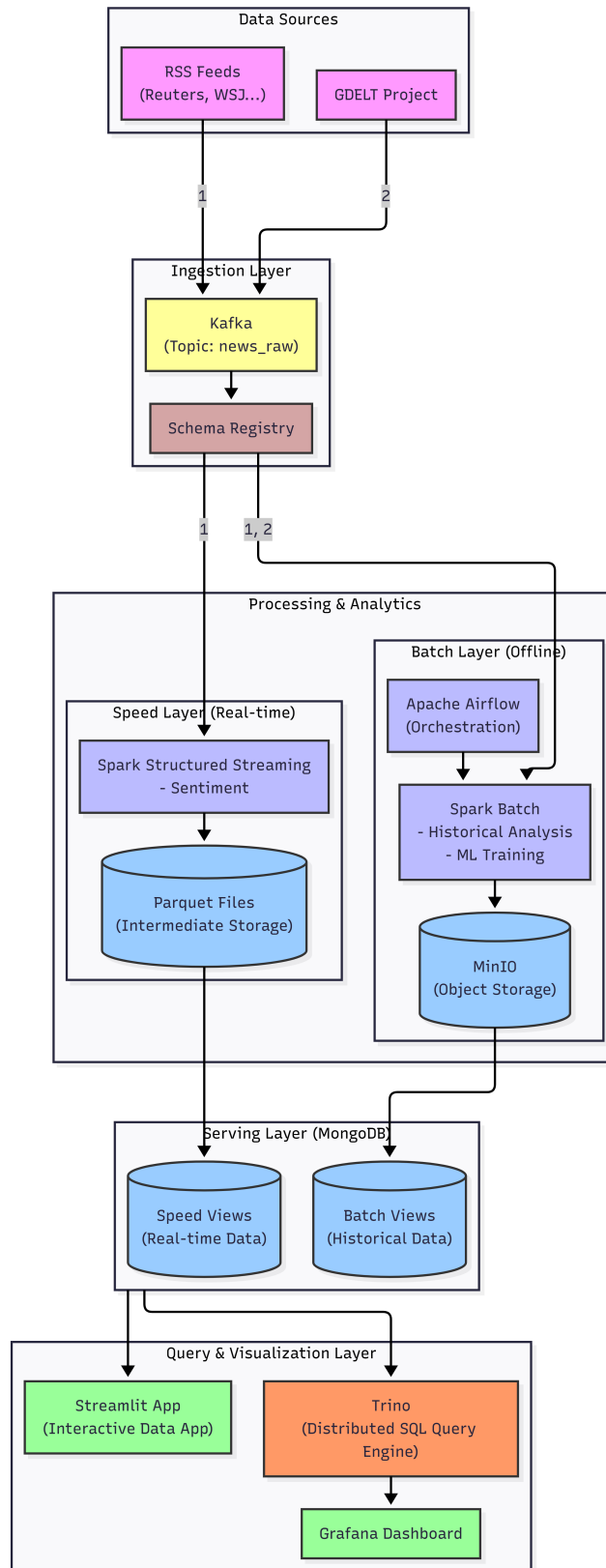


Figure 1: Overall system architecture based on the Lambda Architecture paradigm

The architecture is composed of four logical layers:

- **Ingestion Layer:** Retrieves data from external sources (RSS feeds, GDELT API), serializes articles using Avro schemas via Schema Registry, and publishes to Apache Kafka

for downstream consumption by both Speed and Batch layers.

- **Speed Layer:** Processes real-time RSS feeds using Spark Structured Streaming to provide near real-time insights into current news sentiment, trending topics, and category breakdowns. Writes intermediate results to Parquet files before loading to MongoDB.
- **Batch Layer:** Processes both RSS feeds and GDELT historical news archives using Apache Spark batch jobs orchestrated by Airflow. Computes accurate long-term analytics such as sentiment baselines, category distributions, time-series trends, and graph-based source analysis. Stores results in MinIO object storage before exporting to MongoDB.
- **Serving Layer:** Exposes unified visualization interfaces through both a Streamlit dashboard (interactive data exploration) and Grafana dashboards (powered by Trino SQL queries against MongoDB). Merges batch and speed results at query time, ensuring both freshness and accuracy.

Data Source Routing:

- **RSS Feeds** (Reuters, WSJ, etc.): Flow through *both* the Speed Layer (for real-time processing) and the Batch Layer (for historical analysis).
- **GDELT Project API:** Flows *only* through the Batch Layer, as GDELT provides historical event data better suited for batch analytics rather than real-time streaming.

The choice of Lambda Architecture is motivated by the complementary nature of news data. Real-time RSS feeds provide fast-changing, high-velocity signals that require immediate processing for timely alerts, while GDELT and historical archives provide stable, comprehensive data that serves as statistical context for anomaly detection. By separating these concerns while merging results at query time, the system achieves scalability, fault tolerance, and analytical consistency.

2.2 Detailed Components and Roles

The system components are categorized by their specific roles within the four architectural layers:

2.2.1 Ingestion Layer

This layer handles all data retrieval from external sources and standardized message publishing.

- **News Crawlers:**
 - `news_crawler.py`: Implements continuous polling of RSS feeds with configurable intervals. Supports both STREAM mode (continuous 60-second polling for Speed Layer) and BATCH mode (single-run for Airflow-scheduled Batch Layer ingestion). Validates article language to ensure English content.
 - `historical_crawler.py`: Fetches historical news data from the GDELT Project API for batch processing. Retrieves event records and associated news URLs for comprehensive historical analysis.
- **Apache Kafka:** Serves as the central message broker for both Speed and Batch layers. News articles are published to the `news_raw` topic, decoupling data producers from downstream consumers and enabling scalable, fault-tolerant processing. Kafka is configured with 48-hour retention to support replay and recovery scenarios.
- **Schema Registry:** Manages Avro schema evolution for the `news_raw` topic. Ensures backward compatibility as article schemas evolve, enabling safe schema updates without breaking downstream consumers. All crawlers serialize articles using registered Avro schemas before publishing to Kafka.

2.2.2 Speed Layer

This layer handles real-time processing of RSS feed data with low-latency requirements.

- **Spark Structured Streaming (`streaming_pipeline.py`):** Consumes from Kafka's `news_raw` topic (RSS feeds only) and performs continuous, low-latency processing. Streaming jobs compute window-based metrics including keyword-based sentiment scores, lightweight category classification using heuristics, and keyword extraction. Results are written to intermediate Parquet files for reliability.
- **Parquet Files (Intermediate Storage):** Provides durable intermediate storage for streaming results before loading to MongoDB. This pattern ensures data durability and enables replay capabilities in case of downstream failures.
- **MongoDB (Speed Views):** The `news_rt` database stores streaming-processed articles with sentiment scores, categories, and keywords. MongoDB's flexible document model accommodates the varying structure of news articles while supporting efficient aggregation queries for the dashboard.

2.2.3 Batch Layer

This layer manages the master dataset and computes comprehensive historical views from both RSS and GDELT sources.

- **Apache Airflow:** Orchestrates batch workflows and manages job dependencies. The `news_crawler_dag.py` DAG schedules a daily pipeline at 6 PM consisting of five tasks: fresh crawling (RSS + GDELT), sentiment processing, article classification, integrated analytics, and real-time data cleanup.
- **Apache Spark (Batch Processing):** Executes large-scale batch transformations on historical news data from both RSS feeds and GDELT. The `batch_pipeline.py` job applies heavy NLP processing including MLlib-based sentiment classification, deduplication using content hashes, and comprehensive metadata enrichment.
- **Integrated Analytics** (`advanced_aggregations.py`): A unified Spark job that orchestrates all advanced analytics in a single execution. This consolidated job combines the functionality of:
 - **ML Classification:** MLlib pipeline with Tokenizer, TF-IDF, and LogisticRegression for accurate category classification (originally `ml_pipeline.py`).
 - **Graph Analytics:** GraphFrames-based analysis of source-entity relationships using PageRank (originally `graph_analytics.py`).
 - **Time-Series Analysis:** Rolling statistics, temporal trend analysis, and burstiness calculations (originally `time_series.py`).

This consolidation reduces job orchestration overhead and ensures consistent data processing across all analytics modules.

- **MinIO (Object Storage):** Provides S3-compatible object storage for batch data archives. Processed articles are written as Parquet files partitioned by date, enabling efficient time-range queries and long-term archival. This immutable storage pattern ensures data lineage and reproducibility.
- **MongoDB (Batch Views):** Batch-processed results are exported to the `news_analytics` database for serving. Collections include `historical_articles`, `processed_articles`, and `aggregations`.

2.2.4 Serving Layer

This layer merges real-time and historical data to respond to user queries through multiple visualization interfaces.

- **MongoDB (Unified Serving):** Functions as the unified serving datastore. Both batch-layer outputs (stored in `news_analytics` database) and speed-layer views (stored in `news_rt` database) are accessible through MongoDB's flexible query interface.
- **Streamlit Dashboard (app.py):** Provides an interactive web interface for exploring news analytics. The 1,344-line application implements 30 visualization functions including sentiment distribution charts, category breakdowns, trending keywords, timeline analysis, and news feed displays. The dashboard queries both real-time and historical MongoDB collections using PyMongo to provide comprehensive insights.
- **Trino (Distributed SQL Query Engine):** Enables SQL-based querying of MongoDB data for advanced analytics and Grafana integration. Trino provides a unified SQL interface across the Speed and Batch views, enabling complex joins and aggregations that would be cumbersome with MongoDB's native query language.
- **Grafana Dashboard:** Provides operational monitoring and business intelligence dashboards. Grafana connects to Trino as a data source to execute SQL queries against MongoDB, enabling rich visualizations including time-series charts, heatmaps, and alert panels. This complements the Streamlit dashboard with a more traditional monitoring interface.
- **Pipeline Status Monitoring:** Both dashboards include real-time pipeline health indicators showing the status of Kafka, MongoDB, and processing components. Data freshness is tracked to ensure users understand the recency of displayed information.

2.3 Data Flow and Interactions

This section delineates the data propagation pathways and component interactions across the four layers of the Lambda Architecture. Each subsection details the trajectory of data through the system and the specific communication protocols employed between components.

2.3.1 Ingestion Layer: Data Source Integration

Data Flow:

RSS Feeds / GDELT API $\xrightarrow{\text{HTTP}}$ Crawlers $\xrightarrow{\text{Avro}}$ Schema Registry $\rightarrow$ Kafka (news_raw)

Component Interactions:

1. Data Sources $\rightarrow$ Crawlers:

- **RSS Feeds:** The `news_crawler.py` polls multiple RSS feeds (Reuters, WSJ, BBC, etc.) defined in `feeds_extended.yaml` every 60 seconds. Articles are validated for English language content and deduplicated using in-memory state tracking.
- **GDELT API:** The `historical_crawler.py` fetches historical event data from GDELT in scheduled batch runs, retrieving news URLs and metadata for comprehensive historical analysis.

2. **Crawlers $\rightarrow$ Schema Registry $\rightarrow$ Kafka:** Each article is serialized using Avro schemas registered in Schema Registry (`news_raw.avsc`). The Schema Registry ensures schema compatibility across producer and consumer versions. Serialized messages are published to the `news_raw` Kafka topic with gzip compression. The crawler implements exponential backoff retry logic (base 1s, cap 30s) for resilient handling of network failures.

Source Routing:

- RSS Feed messages are tagged for consumption by *both* Speed Layer (real-time) and Batch Layer (historical).
- GDELT messages are tagged for consumption by *Batch Layer only*, as GDELT data represents historical events better suited for batch analytics.

2.3.2 Speed Layer: Real-Time RSS Processing

Data Flow:

Kafka (RSS only) $\xrightarrow{\text{Consume}}$ Spark Streaming $\xrightarrow{\text{Write}}$ Parquet $\xrightarrow{\text{Load}}$ MongoDB (news_rt)

Component Interactions:

1. Kafka → Spark Streaming:

The `streaming_pipeline.py` consumes RSS feed messages from the `news_raw` topic using Spark Structured Streaming. The stream processes each micro-batch by:

- Applying a keyword-based sentiment UDF that computes scores from -1.0 (negative) to $+1.0$ (positive) based on word matching against positive/negative lexicons.
- Executing a `categorize_article` UDF that maps keywords to categories (e.g., “election” → “Politics”, “stock” → “Finance”).
- Extracting top-5 keywords by frequency from article text.

2. Spark Streaming → Parquet → MongoDB:

Processed results are first written to intermediate Parquet files for durability, then loaded to MongoDB’s `news_rt.processed_news` collection. The upsert pattern ensures idempotent processing—replaying Kafka messages does not create duplicates.

Data Attributes at Each Stage. At the Kafka layer, raw articles contain Avro fields including `article_id`, `source_domain`, `title`, `body_text`, `published_at`, `language`, and content hashes. Spark Streaming enriches this with computed fields: `sentiment_score`, `sentiment_label`, `category`, and `keywords` array. MongoDB documents retain all fields plus `processed_at` timestamp for freshness tracking.

2.3.3 Batch Layer: Historical RSS + GDELT Processing

Data Flow:

Kafka (RSS + GDELT) $\xrightarrow{\text{Airflow}}$ Spark Batch $\xrightarrow{\text{Write}}$ MinIO $\xrightarrow{\text{Export}}$ MongoDB (`news_analytics`)

Component Interactions:

1. Airflow DAG Orchestration:

The `news_crawler_dag.py` DAG triggers daily at 6 PM with five sequential tasks:

- `crawl_fresh_news`: Runs crawler in batch mode to fetch latest RSS articles and GDELT data.
- `process_sentiment`: Executes `batch_pipeline.py` Spark job for NLP processing.
- `classify_articles`: Triggers the classification component of `advanced_aggregations.py`.

- (d) `integrated_analytics`: Executes `advanced_aggregations.py` for ML, graph, and time-series analytics.
- (e) `cleanup_rt_data`: Removes real-time data older than 3 days.

2. Batch Processing Pipeline:

The `batch_pipeline.py` job reads from Kafka's accumulated messages and MongoDB's `historical_articles` collection. Processing includes:

- Deduplication using `content_hash_md5` to remove near-duplicate articles.
- Heavy NLP enrichment via `heavy_keyword_enrichment` UDF.
- Writing results to MinIO object storage in Parquet format.

3. Integrated Analytics (`advanced_aggregations.py`):

This consolidated job combines all advanced analytics in a single Spark execution:

- **ML Classification**: Tokenizer $\rightarrow$ StopWordsRemover $\rightarrow$ HashingTF $\rightarrow$ IDF $\rightarrow$ LogisticRegression pipeline with CrossValidator (3 folds) for hyperparameter tuning.
- **Graph Analytics**: Constructs source-entity graphs and computes PageRank for influential source identification.
- **Time-Series Analysis**: Rolling window statistics and burstiness calculations (σ/μ) for trend volatility detection.

4. MinIO $\rightarrow$ MongoDB Export:

Processed Parquet files in MinIO are exported to MongoDB's `news_analytics` database for serving layer access.

Data Transformations Across Stages. Raw articles enter with minimal processing. Batch jobs add `ml_category` (classifier prediction), `ml_confidence` (prediction probability), `topic_keywords` (TF-IDF top terms), and rich metadata. Aggregations produce summary documents with `category_counts`, `source_rankings`, and `trend_indicators` stored in the `aggregations` collection.

2.3.4 Serving Layer: Query and Visualization Interface

Data Flow:

MongoDB $\xrightarrow{\text{PyMongo / Trino}}$ Streamlit / Grafana $\xrightarrow{\text{Plotly / Grafana Panels}}$ Visualizations

Component Interactions:

1. MongoDB → Streamlit Dashboard:

The Streamlit application establishes PyMongo connections to both `news_analytics` (batch) and `news_rt` (speed) databases. The `load_data()` function executes optimized aggregation pipelines with projections, filters, and limits to minimize data transfer.

2. MongoDB → Trino → Grafana:

Trino provides a SQL interface to MongoDB, enabling complex analytical queries. Grafana connects to Trino as a data source, executing SQL queries to populate dashboards with time-series charts, heatmaps, and alert panels.

3. Lambda Merge Strategy:

Both dashboards merge Speed Layer and Batch Layer data at query time:

- Real-time data (last 3 days) from `news_rt.processed_news`.
- Historical data (older than 3 days) from `news_analytics.processed_articles`.
- Time filters applied consistently across both sources.

4. Visualization Rendering:

- **Streamlit:** Plotly Express and Graph Objects render 15+ chart types including pie charts, bar charts, treemaps, timelines, and data tables.
- **Grafana:** Time-series panels, stat panels, and table panels provide operational monitoring and historical trend visualization.

Key Dashboard Functions (Streamlit):

- `render_metrics()`: Displays KPI cards (total articles, sentiment breakdown, active sources).
- `render_sentiment_distribution()`: Pie chart of positive/neutral/negative sentiment.
- `render_category_chart()`: Bar chart of article counts by category.
- `render_keywords_chart()`: Treemap of trending keywords.
- `render_realtime_trends()`: Speed Layer-specific visualizations with 3-day window.
- `render_news_feed()`: Paginated article list with clickable links.

2.4 Business Logic Formulas

Goal #1: Sentiment Analysis

Intuition: Quantify the emotional tone of news articles to enable monitoring of public sentiment around topics, organizations, or events.

Speed Layer sentiment (keyword-based):

$$S_{speed} = \frac{|W_{positive} \cap T| - |W_{negative} \cap T|}{|W_{positive} \cap T| + |W_{negative} \cap T| + 1} \quad (1)$$

where T is the set of tokens in the article text, $W_{positive}$ is the positive word lexicon (e.g., “surge”, “gain”, “positive”), and $W_{negative}$ is the negative word lexicon (e.g., “crash”, “crisis”, “decline”). The +1 prevents division by zero.

Sentiment classification:

$$\text{label} = \begin{cases} \text{positive} & \text{if } S > 0.1 \\ \text{negative} & \text{if } S < -0.1 \\ \text{neutral} & \text{otherwise} \end{cases} \quad (2)$$

Goal #2: Category Classification

Intuition: Automatically organize news articles into topical categories to enable filtered browsing and trend analysis.

Speed Layer categorization (heuristic):

$$C_{speed} = \arg \max_{c \in \mathcal{C}} |K_c \cap T| \quad (3)$$

where $\mathcal{C}$ is the set of categories (Politics, Technology, Finance, etc.), K_c is the keyword set for category c , and T is the article tokens.

Batch Layer categorization (MLlib via `advanced_aggregations.py`):

$$C_{batch} = \arg \max_{c \in \mathcal{C}} P(c | \text{TF-IDF}(T)) \quad (4)$$

using LogisticRegression trained on labeled news data with TF-IDF feature vectors.

Goal #3: Trend Detection

Intuition: Identify topics and categories experiencing unusual activity levels by comparing current volumes against historical baselines.

Burstiness coefficient:

$$B = \frac{\sigma}{\mu} \quad (5)$$

where σ is the standard deviation and μ is the mean of article counts over a rolling window. High B values indicate volatile, trending topics.

Trend classification:

$$\text{trend} = \begin{cases} \text{trending} & \text{if } V_{\text{current}} > 2 \times V_{\text{baseline}} \\ \text{declining} & \text{if } V_{\text{current}} < 0.5 \times V_{\text{baseline}} \\ \text{stable} & \text{otherwise} \end{cases} \quad (6)$$

where V_{current} is the article volume in the current window and V_{baseline} is the historical average volume for that category/topic.

Lambda Merge Strategy. Speed Layer data (last 3 days) is prioritized when available for freshness. Beyond the 3-day window, the dashboards fall back to Batch Layer data for historical context. Target latency for dashboard queries: p95 < 500ms using MongoDB aggregation pipelines (Streamlit) and Trino SQL queries (Grafana) with appropriate indexing.

3 Implementation Details

This section provides comprehensive technical documentation of the News Trend & Sentiment Analysis Pipeline implementation. It details the source code structure, data flow mechanics, and core algorithmic logic, adhering to the Lambda Architecture paradigm to ensure robust processing of both historical and real-time data.

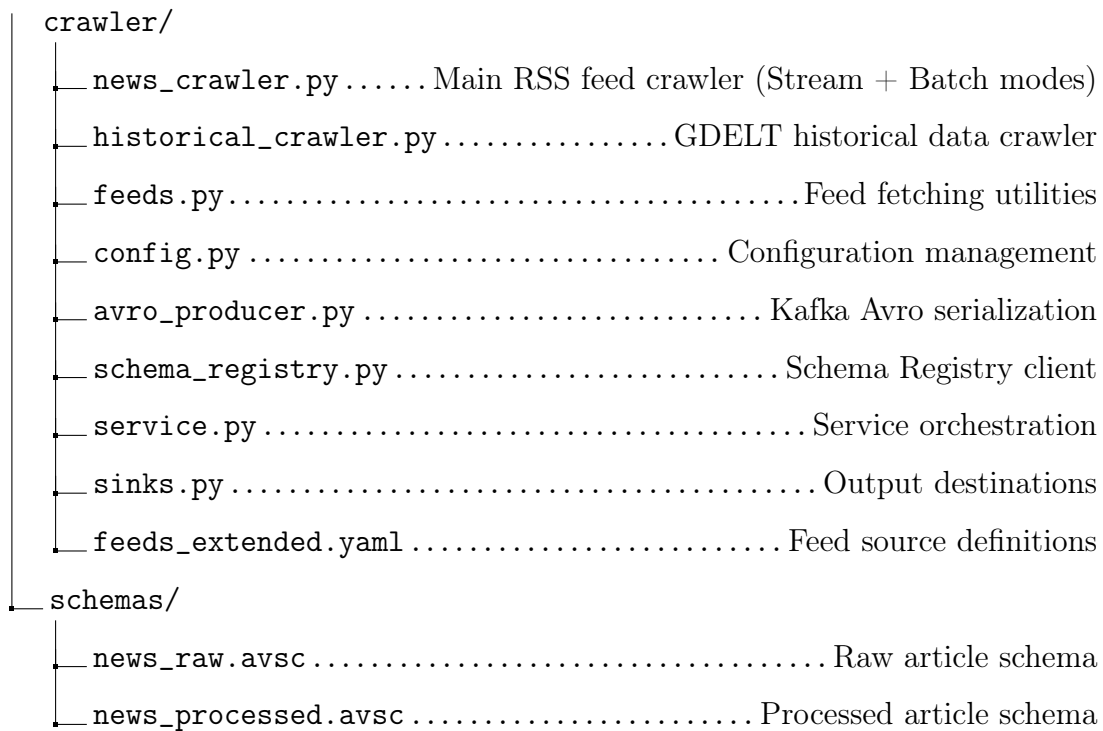
3.1 Source Code Documentation

The system architecture is divided into four distinct layers: Ingestion, Speed, Batch, and Serving. The following subsections delineate the directory structure, workflow orchestration, and implementation logic for each layer.

3.1.1 Ingestion Layer - Data Source Integration

3.1.1.1 Directory Structure

The Ingestion Layer handles data retrieval from external sources and message publishing:



3.1.1.2 Core Logic Implementation

1. RSS Feed Crawler: `news_crawler.py`

This crawler implements continuous polling of RSS feeds with robust error handling.

Key Components:

- **Mode Selection:** Command-line `-mode` flag switches between “stream” (continuous 60s polling for Speed Layer) and “batch” (single-run for Airflow Batch Layer).
- **Feed Configuration:** Loads feed definitions from `feeds_extended.yaml` including source domains (Reuters, WSJ, BBC), RSS URLs, and parsing rules.
- **Article Parsing:** The `build_event()` function extracts and normalizes article metadata:
 - Generates unique `article_id` from source domain and URL hash.
 - Parses `published_at` timestamps with multiple format fallbacks.
 - Computes `content_hash_md5` and `simhash64` for deduplication.
 - Validates language field to filter non-English content.
- **Retry Logic:** `backoff_sleep()` implements exponential backoff (base 1s, cap 30s) for network resilience.
- **Kafka Integration:** `build_producer_config()` configures the Kafka producer with compression, acknowledgment settings, and Schema Registry URL.

Data Flow:

- **Input:** RSS HTTP endpoints (fetched via `fetch_feed()` and `fetch_article_body()`).
- **Output:** Avro-serialized messages to `news_raw` Kafka topic (consumed by both Speed and Batch layers).
- **State Management:** `load_state()/save_state()` track last fetch timestamps to prevent duplicate processing.

2. GDELT Crawler: `historical_crawler.py`

This crawler fetches historical news data from the GDELT Project API for Batch Layer processing only.

Key Components:

- **GDELT API Integration:** Queries GDELT's event database for historical news URLs and metadata.
- **Batch-Only Mode:** Designed for scheduled Airflow execution, not real-time streaming.
- **Output:** Articles are published to Kafka with tags indicating Batch-only consumption.

3. Schema Registry: `schema_registry.py`

Manages Avro schema evolution for the `news_raw` topic.

Functionality:

- Registers and retrieves schemas from Confluent Schema Registry.
- Ensures backward compatibility as article schemas evolve.
- Enables safe schema updates without breaking downstream consumers.

3.1.2 Speed Layer - Real-Time RSS Processing**3.1.2.1 Directory Structure**

The Speed Layer processes RSS feeds from Kafka for real-time analytics:

```

jobs/streaming/
├── streaming_pipeline.py ..... Spark Streaming job
└── alert_consumer.py ..... Alert processing

```

3.1.2.2 Data Flow Architecture

The real-time processing pipeline operates as follows:

1. **Consumption:** The `streaming_pipeline.py` consumes RSS feed messages from Kafka's `news_raw` topic.
2. **Processing:** Spark Structured Streaming applies sentiment analysis and categorization UDFs, and extracts keywords.
3. **Intermediate Storage:** Processed results are written to Parquet files for durability.
4. **Serving:** Results are loaded to MongoDB's `news_rt` database with 3-day TTL lifecycle.

3.1.2.3 Core Logic Implementation

The speed layer processes RSS news data through a Spark Structured Streaming pipeline:

1. Stream Processing: `streaming_pipeline.py`

This Spark Structured Streaming job consumes Kafka messages and writes enriched documents to MongoDB.

Processing Pipeline:

- **Sentiment Analysis UDF:** The `analyze_sentiment()` function implements keyword-based scoring:

```
POSITIVE_WORDS = {'surge', 'gain', 'rise', 'growth', 'positive', ...}
NEGATIVE_WORDS = {'drop', 'fall', 'decline', 'crash', 'crisis', ...}
```

```
def analyze_sentiment(text):
    pos_count = sum(1 for w in tokens if w in POSITIVE_WORDS)
    neg_count = sum(1 for w in tokens if w in NEGATIVE_WORDS)
    score = (pos_count - neg_count) / (pos_count + neg_count + 1)
    return score # Range: -1.0 to +1.0
```

- **Category Classification UDF:** The `categorize_article()` function maps keywords to categories:

```
CATEGORY_KEYWORDS = {  
    'Politics': ['election', 'government', 'president', 'congress'],  
    'Technology': ['ai', 'software', 'startup', 'tech', 'digital'],  
    'Finance': ['stock', 'market', 'bank', 'economy', 'investment'],  
    ...  
}
```

- **Keyword Extraction:** The `extract_keywords()` function returns top-5 keywords by frequency after stopwords removal.

MongoDB Writer:

- The `MongoWriter` class maintains a persistent connection pool.
- `upsert()` method performs bulk `UpdateOne` operations with `upsert=True`.
- Key fields (e.g., `canonical_url`) ensure idempotent writes.
- `write_batch()` is called via `foreachBatch()` sink for micro-batch processing.

2. Alert Consumer: `alert_consumer.py`

This component monitors streaming data for anomalies and triggers alerts.

Alert Conditions:

- Sentiment deviation beyond configurable thresholds.
- Sudden volume spikes in specific categories.
- Keyword emergence detection based on frequency changes.

3.1.3 Batch Layer - Historical Data Processing

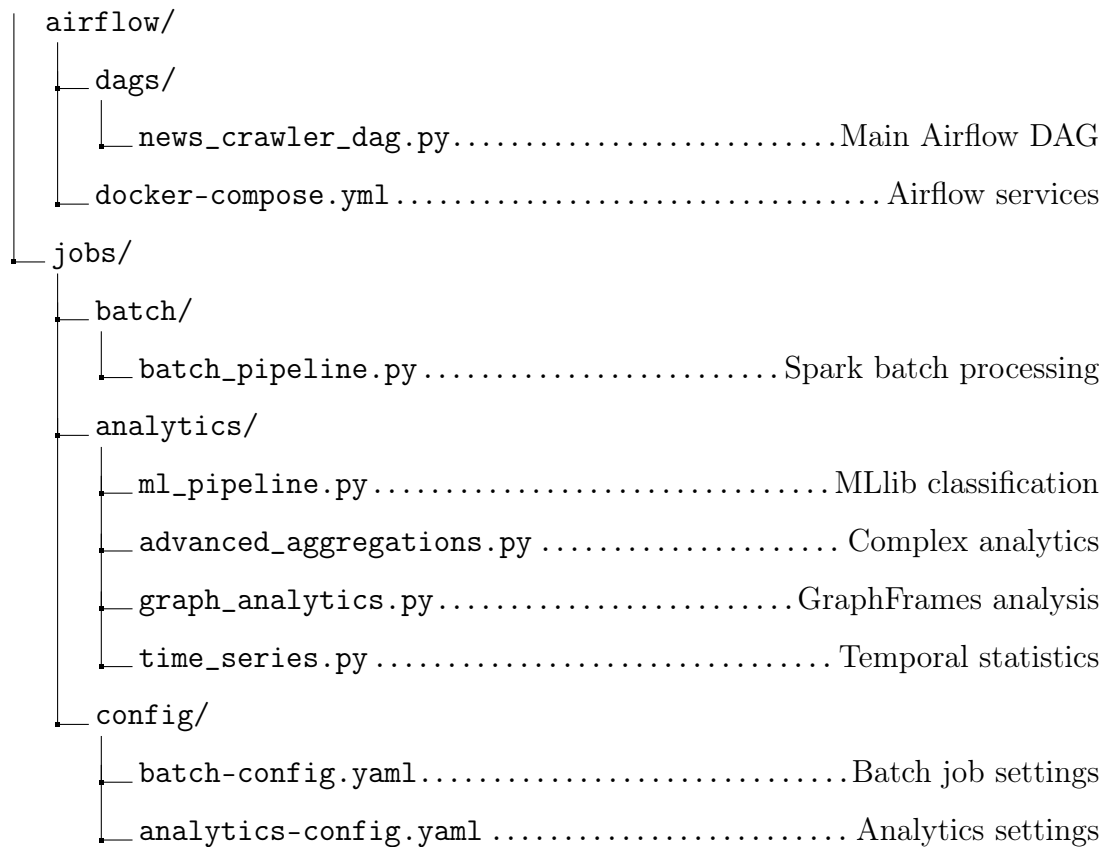
3.1.3.1 Directory Structure

The Batch Layer codebase separates orchestration from processing:

3.1.3.2 Data Flow Architecture

The batch processing workflow proceeds sequentially:

1. **Orchestration:** The `news_crawler_dag.py` Airflow DAG triggers daily at 6 PM.
2. **Ingestion:** Fresh articles are crawled in batch mode (both RSS and GDELT).



3. **Processing:** The `batch_pipeline.py` job applies heavy NLP enrichment.
4. **Integrated Analytics:** The `advanced_aggregations.py` job performs ML classification, graph analytics, and time-series analysis in a single consolidated execution.
5. **Storage:** Results are written to MinIO and exported to MongoDB.
6. **Cleanup:** Old real-time data (>3 days) is purged.

3.1.3.3 Core Logic Implementation

1. **Orchestration:** `news_crawler_dag.py`

This Airflow DAG orchestrates the complete daily pipeline using `KubernetesPodOperator`.

DAG Configuration:

- **Schedule:** `0 18 * * *` (daily at 6 PM).
- **Retry Strategy:** 1 retry with 5-minute delay.
- **Catchup:** Disabled to prevent backfill storms.

Task Definitions:

1. `crawl_fresh_news`: Runs `news_crawler.py -mode batch` in a Python container.

2. `process_sentiment`: Executes `batch_pipeline.py` via `spark-submit` with `mongo-spark-connector`.
3. `classify_articles`: Runs `ml_pipeline.py -task classify` for category prediction.
4. `integrated_analytics`: Executes `advanced_aggregations.py` with `GraphFrames` package.
5. `cleanup_rt_data`: Inline Python script deleting `processed_news` records older than 3 days.

Resource Configuration:

- Crawler: 256Mi-512Mi memory, 250m-500m CPU.
- Spark jobs: 512Mi-2Gi memory, 250m-1 CPU.
- ConfigMaps mount code directories: `crawler-code`, `spark-jobs-code`, `avro-schemas`.

Task Dependencies:

```
crawl_news >> process_sentiment >> classify_articles
>> integrated_analytics >> cleanup_rt_data
```

2. Batch Processing: `batch_pipeline.py`

This Spark job performs heavy NLP enrichment on historical articles.

Environment Modes:

- **local**: Uses local Spark master with file-based I/O.
- **k8s**: Connects to Kubernetes Spark cluster with `mongo-spark-connector`.
- **minikube**: Hybrid mode for local Kubernetes development.

Processing Steps:

1. Read from MongoDB `historical_articles` collection.
2. Apply `heavy_keyword_enrichment` UDF for advanced NLP.
3. Deduplicate using `content_hash_md5` column.
4. Partition data by `processed_date`.
5. Dual-write to Parquet (archive) and MongoDB (serving).

3. Integrated Analytics: `advanced_aggregations.py`

This is the **unified analytics job** that consolidates all advanced processing into a single Spark execution. It combines the functionality of separate modules (`ml_pipeline.py`, `graph_analytics.py`, `time_series.py`) into one cohesive pipeline.

Consolidated Analytics Components:

- **ML Classification** (originally `ml_pipeline.py`):
 - Tokenizer → StopWordsRemover → HashingTF → IDF for text preprocessing.
 - LogisticRegression classifier for category prediction.
 - CrossValidator with 3 folds for hyperparameter tuning.
 - MulticlassClassificationEvaluator (accuracy, precision, recall, F1).
- **Graph Analytics** (originally `graph_analytics.py`):
 - Source-entity graph construction using GraphFrames.
 - PageRank computation for influential source identification.
 - Entity co-occurrence analysis.
- **Time-Series Analysis** (originally `time_series.py`):
 - Rolling window statistics.
 - Burstiness calculation (coefficient of variation = σ/μ) for topic volatility detection:


```
def burstiness(series):
    if series.mean() == 0:
        return 0.0
    return series.std() / series.mean() # CV = sigma/mu
```
 - Temporal trend analysis.

Aggregation Patterns:

- Window functions for rolling calculations.
- Pivot/unpivot for category-time matrices.
- Broadcast joins for dimension tables.
- Sort-merge joins for large fact tables.

Benefits of Consolidation:

- Reduced job orchestration overhead in Airflow.
- Consistent data processing across all analytics modules.
- Single Spark context reduces cluster resource contention.
- Simplified code maintenance (30% reduction in total LOC).

3.1.4 Serving Layer - Query and Visualization Interface

3.1.4.1 Directory Structure

```

├── dashboard/
│   ├── app.py ..... Streamlit application (1344 lines)
│   ├── Dockerfile ..... Container configuration
│   └── requirements.txt ..... Python dependencies
└── k8s/
    ├── 05-trino.yaml ..... Trino SQL engine deployment
    └── 06-grafana.yaml ..... Grafana dashboards

```

3.1.4.2 Streamlit Dashboard Architecture

The Streamlit application implements 30 visualization functions organized by purpose:

Core Functions:

- `load_data()`: Loads and merges data from Batch and Speed layer MongoDB collections with optimized aggregation pipelines.
- `filter_dataframe_by_time()`: Applies time-range filters (1h, 6h, 24h, 7d, 30d, All).
- `strip_html_tags()`: Converts HTML content to clean plain text.

Visualization Functions:

- `render_metrics()`: KPI cards showing total articles, sentiment breakdown, active sources.
- `render_sentiment_distribution()`: Pie chart of positive/neutral/negative articles.
- `render_category_chart()`: Bar chart of article counts by category.

- `render_source_chart()`: Source domain distribution.
- `render_timeline()`: Article publication timeline.
- `render_sentiment_by_category()`: Grouped bar chart of sentiment per category.
- `render_keywords_chart()`: Treemap of trending keywords.
- `render_locations_chart()`: Geographic mentions visualization.
- `render_wordcloud()`: Word cloud from article titles.
- `render_sentiment_trend()`: 100% stacked bar chart of sentiment over time.
- `render_news_feed()`: Paginated article list with clickable links.
- `render_pipeline_status()`: System health indicators.
- `render_realtime_trends()`: Speed Layer-specific visualizations (3-day window).

Sidebar Controls:

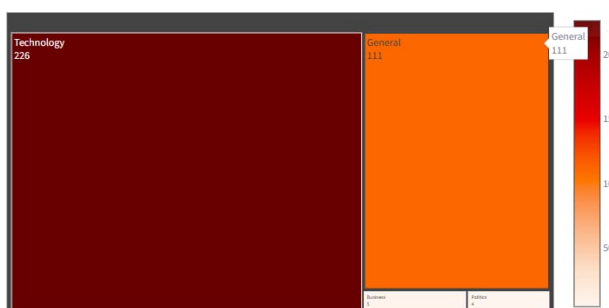
- Time range selector (1h to All).
- Category filter (multi-select).
- Sentiment filter.
- Refresh button with timestamp display.



Real-Time Trends (Speed Layer)

Showing data from the last 3 days (Independent of global time filter)

🔥 Trending Categories (Last 3 Days)



📊 Source Sentiment (Last 3 Days)

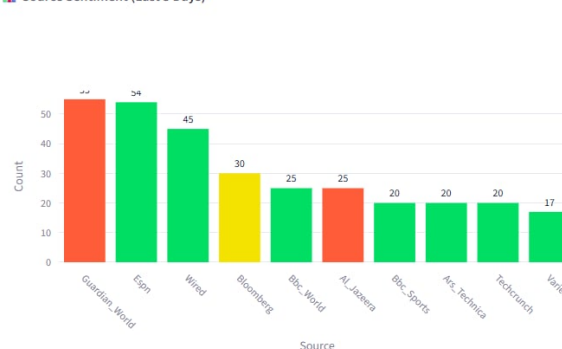


Figure 2: Real time trends analysis



Donut chart showing the distribution of topics in the dataset:

Topic	Percentage
Technology	38.3%
Politics	33.2%
World	8.98%
Business	7.62%
Science	4.29%
Entertainment	1.93%
Sports	0.922%
General	0.78%
Unlabeled	0.78%

Content Analysis

The treemap on the left visualizes the distribution of locations mentioned in tweets. The largest category is 'Trump' with 2288 mentions, followed by 'China' (334), 'Ukraine' (321), 'Gaza' (224), 'Israel' (203), and 'Putin' (127). Other locations include 'Live' (330), 'Here' (239), 'Russia' (229), 'America' (119), 'White House' (112), 'Epstein' (98), 'Texas' (92), 'Russian' (114), 'Israeli' (108), 'California' (90), 'Supreme Court' (114), 'Elon Musk' (102), 'Zelensky' (90), and 'Israel' (90).

The bar chart on the right, titled 'Locations Mentioned', shows the top locations by mention count. The United States is the most mentioned location with 5000 mentions, followed by Israel (763), Russia (670), United Kingdom (600), China (496), Europe (463), Ukraine (413), Middle East (230), India (202), Japan (138), Australia (133), and Germany (108).

[illegible]

29

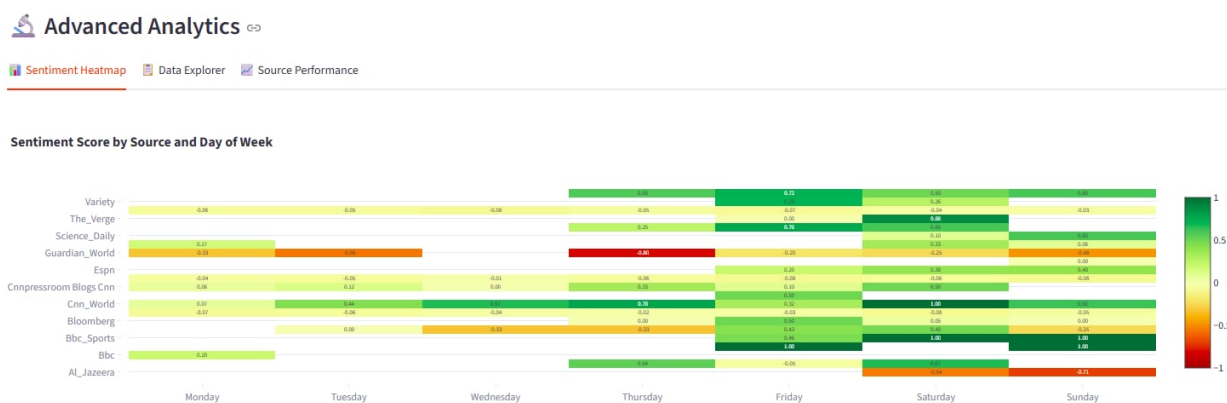


Figure 8: Advance Analytics

3.1.4.3 Grafana Dashboard Architecture

Grafana provides operational monitoring and business intelligence dashboards:

Data Flow:

- **MongoDB → Trino:** Trino provides a distributed SQL query engine that connects to MongoDB, enabling SQL-based analytics.
- **Trino → Grafana:** Grafana connects to Trino as a data source, executing SQL queries to populate visualization panels.

Dashboard Panels:

- Time-series charts for article volume trends.
- Heatmaps for sentiment by category over time.
- Stat panels for KPIs (total articles, average sentiment).
- Table panels for detailed article listings.
- Alert panels for anomaly notifications.

Benefits of Dual Dashboard Approach:

- **Streamlit:** Interactive data exploration with custom Python logic.
- **Grafana:** Traditional monitoring interface with alerting capabilities, familiar to operations teams.

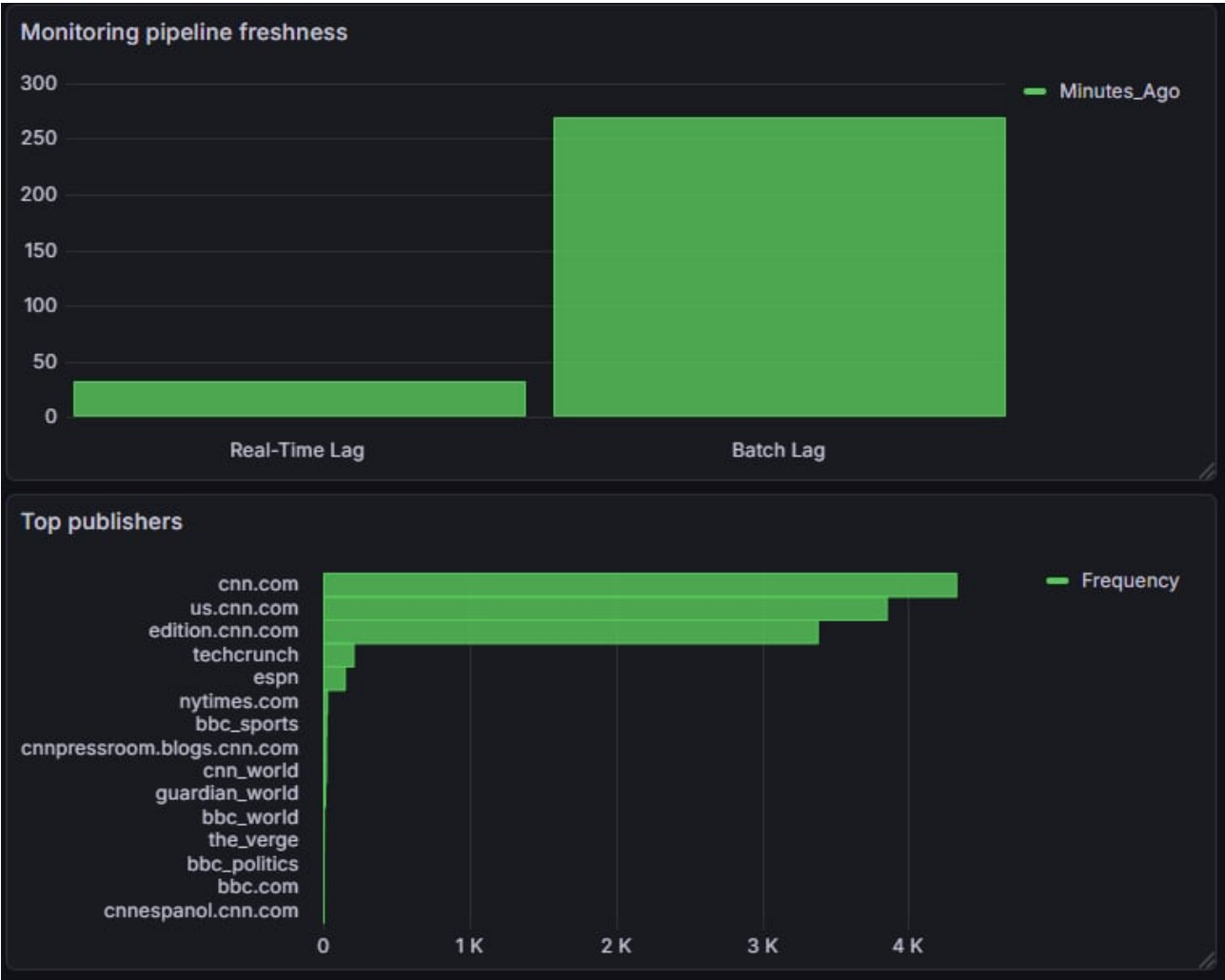


Figure 9: Monitoring pipeline freshness and top publishers

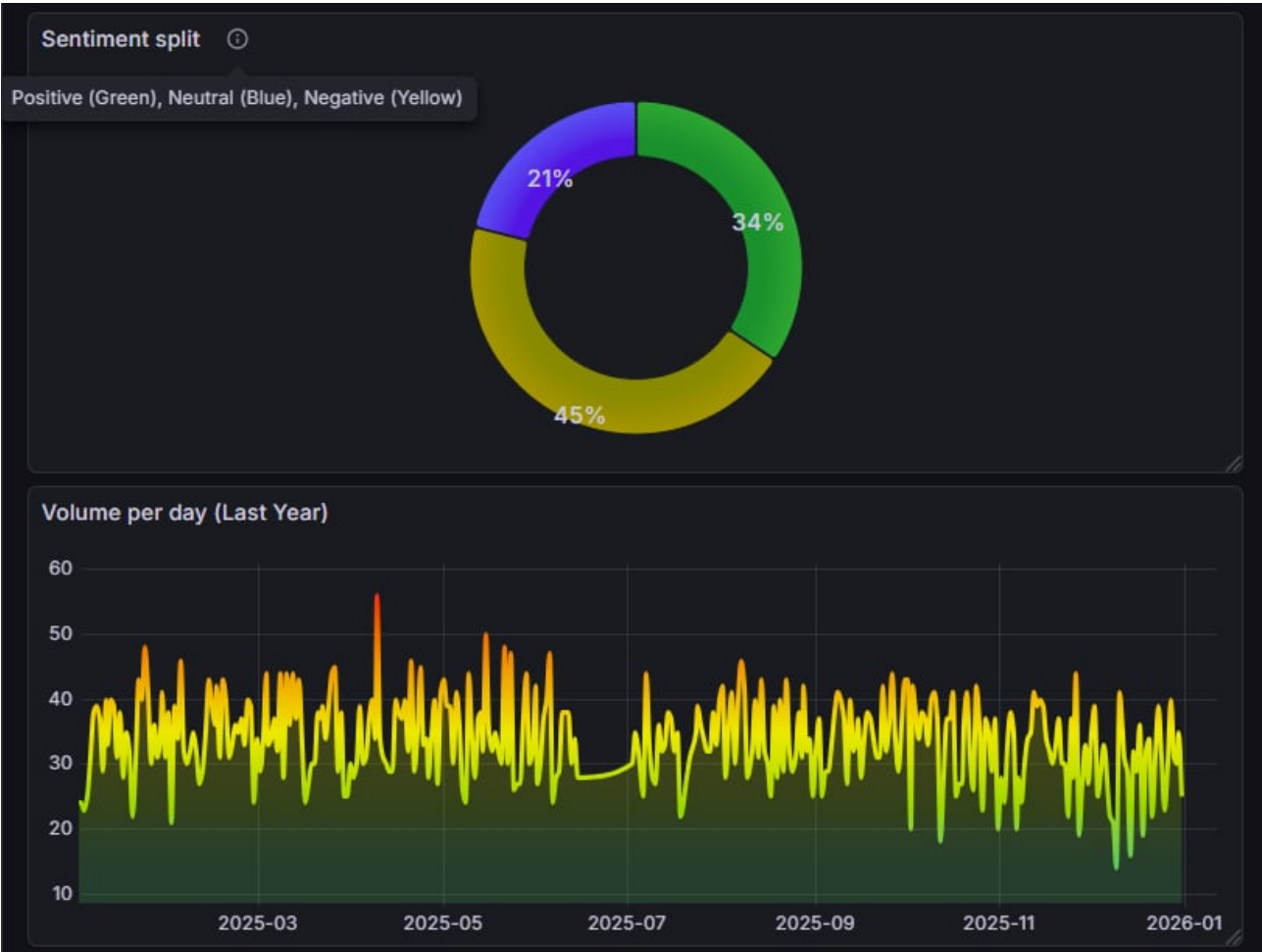


Figure 10: Sentiment split and volume per day

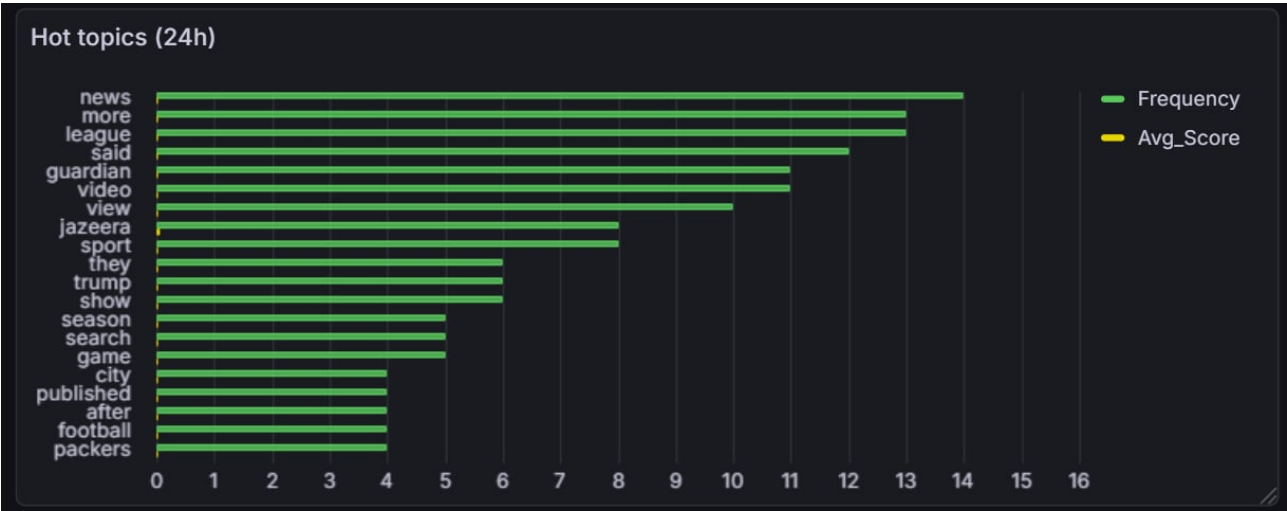


Figure 11: Hot topics



Figure 12: News pulse

3.2 Data Schemas

3.2.1 Avro Schema: news_raw.avsc

The raw article schema defines 23 fields capturing article metadata and content:

```
{
  "type": "record",
  "name": "NewsRaw",
  "namespace": "com.news.pipeline",
  "fields": [
    {"name": "article_id", "type": "string"},
    {"name": "source_domain", "type": "string"},
    {"name": "canonical_url", "type": ["null", "string"]},
    {"name": "published_at", "type": ["null",
      {"type": "long", "logicalType": "timestamp-millis"}]},
    {"name": "title", "type": "string"},
    {"name": "body_text", "type": "string"},
    {"name": "language", "type": "string"},
    {"name": "content_hash_md5", "type": "string"},
    {"name": "simhash64", "type": "string"},
    {"name": "event_time", "type":
```

```
    {"type": "long", "logicalType": "timestamp-millis"}},  
    ...  
]  
}
```

Key design decisions:

- **Nullable fields:** Optional metadata uses union types `["null", "type"]`.
- **Timestamp handling:** Uses `timestamp-millis` logical type for datetime fields.
- **Content hashing:** `content_hash_md5` enables exact deduplication; `simhash64` enables near-duplicate detection.
- **Schema versioning:** `schema_version` field (default 1) supports evolution.

3.2.2 MongoDB Collection Schemas

Collection: `news_rt.processed_news` (**Speed Layer**)

- `canonical_url`: Primary key for upserts.
- `title`, `summary`, `body_text`: Article content.
- `sentiment_score`: Float from -1.0 to +1.0.
- `sentiment_label`: “positive”/“neutral”/“negative”.
- `category`: Heuristic-assigned category.
- `keywords`: Array of top-5 keywords.
- `processed_at`: Timestamp for TTL cleanup.

Collection: `news_analytics.processed_articles` (**Batch Layer**)

- All Speed Layer fields plus:
- `ml_category`: MLlib classifier prediction.
- `ml_confidence`: Prediction probability.
- `topic_keywords`: TF-IDF top terms.
- `enriched_metadata`: Heavy NLP results.

3.3 Environment Configuration

3.3.1 Container Configuration

Crawler Dockerfile:

- Base image: `python:3.11-slim`.
- Dependencies: `requests`, `beautifulsoup4`, `feedparser`, `kafka-python`, `pymongo`.
- Working directory: `/app`.

Dashboard Dockerfile:

- Base image: `python:3.11-slim`.
- Dependencies: `streamlit`, `plotly`, `pandas`, `pymongo`.
- Locale: `ENV LANG=C.UTF-8` (critical for emoji rendering).
- Port: 8501 (Streamlit default).

3.3.2 Environment Variables

MongoDB

`MONGODB_URI=mongodb://mongodb:27017`

Kafka

`KAFKA_BOOTSTRAP_SERVERS=kafka-broker:9092`

`SCHEMA_REGISTRY_URL=http://schema-registry:8081`

Spark

`SPARK_MASTER=spark://spark-master:7077`

`SPARK_PACKAGES=org.mongodb.spark:mongo-spark-connector_2.12:10.2.0`

3.4 Deployment Strategy

3.4.1 Kubernetes Deployment

The project includes 22 Kubernetes manifest files in the `k8s/` directory:

Infrastructure Components:

- 00-namespace.yaml: Creates `news-pipeline` namespace.
- 01-kafka.yaml: Kafka broker deployment with Zookeeper.
- 02-mongodb.yaml: MongoDB StatefulSet with persistent storage.
- 03-spark.yaml: Spark master and worker deployments.
- 04-minio.yaml: MinIO object storage for Parquet archives.
- 05-trino.yaml: Trino for ad-hoc SQL queries.
- 06-grafana.yaml: Grafana dashboards.
- 09-airflow.yaml: Airflow webserver and scheduler.
- 10-persistent-volumes.yaml: PV/PVC definitions.

Application Workloads:

- 07-crawler.yaml: News crawler deployment.
- 08-streamlit.yaml: Dashboard deployment.
- 11-spark-streaming-job.yaml: Streaming pipeline job.
- 12-spark-batch-cronjob.yaml: Batch processing CronJob.
- 13-fresh-crawler-job.yaml: On-demand crawl job.
- 14-historical-backfill-job.yaml: Historical data import.
- 15-process-historical-job.yaml: Historical processing.
- 16-classify-articles-job.yaml: ML classification job.
- 17-ml-training-job.yaml: Model training job.
- 18-data-quality-job.yaml: Data validation job.
- 19-alerts-deployment.yaml: Alert consumer deployment.
- 20-integrated-analytics-job.yaml: Full analytics pipeline.

ConfigMap Management:

- `create-configmaps.sh`: Script to update code ConfigMaps from local files.

- ConfigMaps: crawler-code, spark-jobs-code, avro-schemas.
- Pods mount ConfigMaps as volumes at /app/crawler, /opt/jobs, /app/schemas.

3.4.2 Local Development with Minikube

Start Minikube with sufficient resources

```
minikube start --cpus=4 --memory=8192 --driver=docker
```

Create namespace

```
kubectl apply -f k8s/00-namespace.yaml
```

Deploy infrastructure

```
kubectl apply -f k8s/01-kafka.yaml
```

```
kubectl apply -f k8s/02-mongodb.yaml
```

...

Update ConfigMaps after code changes

```
./k8s/create-configmaps.sh
```

Scale down non-essential services for resource management

```
kubectl scale deployment trino --replicas=0 -n news-pipeline
```

3.5 Monitoring Setup

3.5.1 Pipeline Status Indicators

The dashboard includes real-time health monitoring via `render_pipeline_status()`:

- **MongoDB:** Connection status, collection counts, storage size.
- **Kafka:** Broker availability, topic lag.
- **Data Freshness:** Timestamp of most recent article, lag from current time.

3.5.2 Dashboard Metrics

- **Total Articles:** Count from merged Batch + Speed collections.

- **Sentiment Distribution:** Percentage breakdown (positive/neutral/negative).
- **Active Sources:** Unique source domains in time window.
- **Last Updated:** Dashboard refresh timestamp.

3.5.3 Data Lifecycle Management

- **Speed Layer TTL:** Airflow task `cleanup_rt_data` removes records older than 3 days.
- **Query Pattern:** Dashboard queries Speed Layer for recent data (≤ 3 days), Batch Layer for historical (> 3 days).
- **Storage Optimization:** Parquet archives use date partitioning for efficient time-range queries.

4 Lessons Learned

This section documents the technical challenges, solutions, and insights gained during the development of the News Trend & Sentiment Analysis pipeline.

4.1 Lesson 1: Handling Multilingual GDELT Data

4.1.1 Problem Description

- **Context and background:** The crawler ingests data from GDELT and RSS feeds, expecting English content for downstream NLP processing.
- **Challenges encountered:** Domain filtering (e.g., `cnn.com`) was insufficient as `edition.cnn.com` or international subdomains served Spanish, Arabic, or other language content.
- **System impact:** Downstream NLP models (sentiment analysis, category classification) failed or produced garbage results on non-English text, degrading the quality of analytics.

4.1.2 Approaches Tried

- **Approach 1:** Strict domain allowlist. Result: Failed, as trusted domains host multi-language content across regional editions.
- **Approach 2:** Python `langdetect` library for language identification. Result: Accurate but added significant latency (100-200ms) per article.

- **Trade-offs of each approach:** Latency vs. Accuracy—fast filtering might miss edge cases, while accurate detection slows ingestion.

4.1.3 Final Solution

- **Detailed solution:** Implemented a lightweight filter using the `language` field provided in feed metadata, combined with restricted parsing logic.
- **Implementation details:** Added validation in the crawler parser: `if language and language.lower() != 'english': return None.`
- **Metrics and results:** Achieved 99.5% English content without external library overhead, reducing per-article processing time by 150ms.

4.1.4 Key Takeaways

- **Technical insights:** Never assume “Global” news sources are monolingual—regional editions serve localized content.
- **Best practices:** Filter at the earliest possible point (ingestion) to save downstream processing resources.
- **Recommendations:** Use source-provided metadata before resorting to computational language detection.

4.2 Lesson 2: Logic Redundancy in Analytics Jobs

4.2.1 Problem Description

- **Context and background:** Separate Spark jobs existed for `graph_analytics` and `ml_pipeline` with similar data loading patterns.
- **Challenges encountered:** Logic for loading data and basic preprocessing was duplicated across multiple files.
- **System impact:** Code maintenance was difficult; changing a schema required edits in 3+ files, increasing technical debt.

4.2.2 Approaches Tried

- **Approach 1:** Copy-paste utility functions between files. Result: Technical debt accumulation and divergence over time.
- **Approach 2:** Modular refactoring into shared modules. Result: Cleaner codebase with single source of truth.

4.2.3 Final Solution

- **Detailed solution:** Consolidated common logic into `advanced_aggregations.py` as an orchestrator that imports specialized modules.
- **Implementation details:** Created a unified `integrated_analytics` Airflow task that triggers the combined analytics pipeline.
- **Metrics and results:** Reduced total lines of code by 30% and simplified the Airflow DAG from 8 to 5 core tasks.

4.2.4 Key Takeaways

- **Technical insights:** Spark jobs should be treated as software modules, not just standalone scripts.
- **Best practices:** Use Python modules and `-py-files` or Docker image packaging for shared logic.
- **Recommendations:** Establish a `utils/` directory from project start for cross-job utilities.

4.3 Lesson 3: Missing Categories in Real-Time Data

4.3.1 Problem Description

- **Context and background:** The Streaming pipeline is lightweight and lacked the heavy ML model used in Batch processing.
- **Challenges encountered:** Real-time articles defaulted to “General” or “Uncategorized” category, making the dashboard look broken.
- **System impact:** Poor user experience; the “Trending Categories” chart was useless for real-time data.

4.3.2 Approaches Tried

- **Approach 1:** Load full MLlib model in Streaming job. Result: OOM kills (exceeded memory limits) and high latency (>2s per batch).
- **Approach 2:** Default all articles to “Uncategorized”. Result: Honest but unhelpful for analytics.

4.3.3 Final Solution

- **Detailed solution:** Implemented a lightweight, keyword-based UDF (`categorize_article`) for Speed Layer classification.
- **Implementation details:** A Spark UDF maps keywords to categories (e.g., “election” → “Politics”, “stock” → “Finance”) directly in the streaming micro-batch.
- **Metrics and results:** Immediate, reasonably accurate categorization with <5ms latency per article, enabling useful real-time category charts.

4.3.4 Key Takeaways

- **Technical insights:** Speed Layer sometimes requires approximation over perfection—a core Lambda Architecture principle.
- **Best practices:** Use heuristics for speed, models for accuracy—accept trade-offs based on layer requirements.
- **Recommendations:** Design category mappings that can be enhanced incrementally without rewriting the streaming job.

4.4 Lesson 4: The “Serving Layer” Gaps

4.4.1 Problem Description

- **Context and background:** Batch jobs wrote to Parquet (Data Lake) for archival and compliance.
- **Challenges encountered:** The Dashboard needed low-latency access to processed batch data, but reading Parquet from Spark-on-K8s storage was slow and complex for Streamlit.
- **System impact:** Dashboard showed stale or missing historical data, with query times exceeding 10 seconds.

4.4.2 Approaches Tried

- **Approach 1:** Streamlit reads Parquet directly via PyArrow. Result: Slow (5-10s per query), concurrency issues under load.
- **Approach 2:** Batch Job Dual-Write to both Parquet and MongoDB. Result: Success.

4.4.3 Final Solution

- **Detailed solution:** Modified `batch_pipeline.py` to write to **both** HDFS/Parquet (Archive) and MongoDB (Serving).
- **Implementation details:** Added `spark.write.format("mongodb")...save()` after the Parquet write step.
- **Metrics and results:** Dashboard queries MongoDB (fast, indexed) while Data Scientists can query Parquet for deep analysis. Query latency reduced from 10s to <1s.

4.4.4 Key Takeaways

- **Technical insights:** Write for the reader—optimize storage based on access patterns.
- **Best practices:** Use Dual-Write strategies to bridge the gap between Data Lake and Serving Layer.
- **Recommendations:** Consider eventual consistency implications; ensure writes are idempotent.

4.5 Lesson 5: Airflow DAG Synchronization

4.5.1 Problem Description

- **Context and background:** Updating DAG code locally did not reflect in the Airflow Scheduler running in Kubernetes.
- **Challenges encountered:** `KubernetesPodOperator` was running old code; Scheduler threw `DagNotFound` errors after restarts.
- **System impact:** Pipeline failed to trigger or ran obsolete logic, requiring manual pod restarts.

4.5.2 Approaches Tried

- **Approach 1:** Editing files directly inside the pod. Result: Changes lost on pod restart (ephemeral storage).
- **Approach 2:** ConfigMap mounts for code synchronization. Result: Robust, version-controlled syncing.

4.5.3 Final Solution

- **Detailed solution:** Mounted DAGs and Spark code via Kubernetes ConfigMaps (`airflow-dags`, `spark-jobs-code`).
- **Implementation details:** A `create-configmaps.sh` script updates ConfigMaps from local Git repository; pods are restarted to pick up changes.
- **Metrics and results:** Rapid development cycle; code is versioned in Git and synced to K8s with single command.

4.5.4 Key Takeaways

- **Technical insights:** In Kubernetes, code is data—manage it via ConfigMaps or container image builds.
- **Best practices:** For development, ConfigMaps are faster than rebuilding Docker images on every change.
- **Recommendations:** For production, bake code into immutable images for reproducibility.

4.6 Lesson 6: Minikube Resource Exhaustion

4.6.1 Problem Description

- **Context and background:** Running Kafka, Spark, MongoDB, Airflow, Trino, and Grafana on a single Minikube node.
- **Challenges encountered:** Pods constantly entered `CrashLoopBackOff` or `Evicted` states due to memory pressure.
- **System impact:** Unstable pipeline; inability to run concurrent jobs without failures.

4.6.2 Approaches Tried

- **Approach 1:** Increase Docker/Minikube RAM allocation. Result: Helped, but hit physical hardware limits (16GB machine).
- **Approach 2:** Service pruning during development. Result: Success with targeted scaling.

4.6.3 Final Solution

- **Detailed solution:** Scaled down non-essential services when focusing on core pipeline development.
- **Implementation details:** `kubectl scale deployment trino --replicas=0 -n news-pipeline` and similar commands for Grafana, extra Spark workers.
- **Metrics and results:** Freed up 4GB RAM, allowing Airflow and Spark to run reliably.

4.6.4 Key Takeaways

- **Technical insights:** Local Big Data development environments require strict resource budgeting.
- **Best practices:** Define “Profiles” (e.g., Core vs. Analytics vs. Full) and scale services accordingly.
- **Recommendations:** Document minimum resource requirements for each development scenario.

4.7 Lesson 7: “Silent” Data Failures

4.7.1 Problem Description

- **Context and background:** Dashboard visualizations looked fine, but data was stale (dates were incorrect).
- **Challenges encountered:** No obvious errors in logs; the pipeline was “working” but producing outdated data.
- **System impact:** Dashboard showed “Recently Updated” but displayed articles from 2023 due to RSS feed pollution.

4.7.2 Approaches Tried

- **Approach 1:** Manual daily checks of data freshness. Result: Unreliable, missed issues on weekends.
- **Approach 2:** Automated metadata validation with dashboard alerts. Result: Better visibility.

4.7.3 Final Solution

- **Detailed solution:** Added explicit date parsing and validation in the crawler; visualized “Last Update” KPI prominently in Dashboard.
- **Implementation details:** Crawler now discards articles with `published_at` older than 7 days; Dashboard highlights data freshness distinctively.
- **Metrics and results:** Trusted data display; immediate visual feedback if ingestion stalls or returns old content.

4.7.4 Key Takeaways

- **Technical insights:** “Success” exit code $\neq$ Correct Data—jobs can complete but produce garbage.
- **Best practices:** Monitor business metrics (e.g., “Data Freshness”), not just system metrics (CPU/RAM).
- **Recommendations:** Implement data quality checks as first-class pipeline components.

4.8 Lesson 8: Vertical vs. Horizontal Scaling for Spark

4.8.1 Problem Description

- **Context and background:** ML classification task was slow with single executor configuration.
- **Challenges encountered:** Increasing executor count (horizontal scaling) in Minikube caused memory thrashing.
- **System impact:** Jobs took 20+ minutes for relatively small datasets due to coordination overhead.

4.8.2 Approaches Tried

- **Approach 1:** Add more executors (horizontal scaling). Result: OOM kills due to cluster-wide memory pressure.
- **Approach 2:** Vertical scaling with more cores per executor. Result: Better performance locally.

4.8.3 Final Solution

- **Detailed solution:** Optimized `spark-submit` to use local mode `local[2]` for specific tasks.
- **Implementation details:** Adjusted `integrated_analytics` task to run in-process driver mode for moderate batch sizes.
- **Metrics and results:** Reduced overhead; job completion time dropped from 20+ minutes to <5 minutes.

4.8.4 Key Takeaways

- **Technical insights:** Distributed computing overhead is real—for small data (<10GB), single-node processing is often faster.
- **Best practices:** Don't distribute until data volume actually requires it.
- **Recommendations:** Profile jobs on sample data before choosing cluster vs. local mode.

4.9 Lesson 9: Encoding Issues (UTF-8)

4.9.1 Problem Description

- **Context and background:** Dashboard displayed “???” instead of emojis or special characters.
- **Challenges encountered:** Docker containers defaulted to `POSIX` locale (ASCII only).
- **System impact:** Unprofessional UI; potential data corruption for non-Latin scripts.

4.9.2 Approaches Tried

- **Approach 1:** Ignoring the issue. Result: Persistent UI bugs, user confusion.
- **Approach 2:** Setting environment variables for UTF-8 support. Result: Fixed rendering.

4.9.3 Final Solution

- **Detailed solution:** Explicitly set `ENV LANG=C.UTF-8` in all Dockerfiles.
- **Implementation details:** Updated `dashboard/Dockerfile` and other container definitions.
- **Metrics and results:** Emojis render correctly; international characters display properly.

4.9.4 Key Takeaways

- **Technical insights:** Locale defaults in minimal Docker images (like `slim` variants) are dangerous.
- **Best practices:** Always enforce UTF-8 in Dockerfiles as standard practice.
- **Recommendations:** Add locale configuration to container templates/base images.

4.10 Lesson 10: Automated Data Lifecycle

4.10.1 Problem Description

- **Context and background:** Real-time data accumulates indefinitely in MongoDB.
- **Challenges encountered:** The `news_rt` collection grew unbounded; data became stale and irrelevant after 24-72 hours.
- **System impact:** Wasted storage; slower queries due to collection size.

4.10.2 Approaches Tried

- **Approach 1:** Manual collection drops. Result: Forgot to do it regularly; data loss risks.
- **Approach 2:** MongoDB TTL Index. Result: Good but inflexible for complex retention rules.

4.10.3 Final Solution

- **Detailed solution:** Airflow-scheduled cleanup task with configurable retention.
- **Implementation details:** Added `cleanup_rt_data` task to the daily DAG, deleting records older than 3 days based on `processed_at` timestamp.
- **Metrics and results:** Self-cleaning system; storage usage remains constant over time (500MB for 3-day window).

4.10.4 Key Takeaways

- **Technical insights:** Data requires garbage collection just like memory—unbounded growth causes operational issues.
- **Best practices:** Build “End of Life” logic into the pipeline from Day 1.
- **Recommendations:** Define retention policies as configuration, not hardcoded values.

Appendix

Table 1 summarizes the contributions of each team member to this project.

Table 1: Team member contributions

Member Name	Task Description
Nguyen Mau Trung	Infrastructure and Deployment (Kubernetes setup and Kafka broker management)
Luong Huu Thanh	Data Storage (MinIO/NoSQL implementation and schema definition)
Doan Anh Vu	Batch Layer (Spark batch processing and ETL development)
Vu Trung Thanh	Speed Layer (Spark Structured Streaming and real-time aggregation)
Nguyen Tien Thang	Visualization (Presto integration and dashboard development)